

AIM

To simulate a cloud computing scenario by replacing the default CloudSim broker with a custom Shortest Job First (SJF) scheduling algorithm, thereby understanding how CloudSim can be extended to implement custom resource management and scheduling strategies.

PROBLEM STATEMENT

In CloudSim, the Datacenter Broker acts as an intermediary between users and cloud resources. The broker manages cloudlets, assigns them to virtual machines, and handles scheduling decisions. The default scheduling behavior does not consider the execution length of individual tasks.

In this experiment, a custom SJFDatacenterBroker is implemented by extending the CloudSim DatacenterBroker. The custom broker sorts the submitted cloudlets according to their execution length and schedules the shortest task first.

- Create a CloudSim datacenter.
- Create a virtual machine.
- Create cloudlets with different execution lengths.
- Implement a custom SJF broker.
- Sort cloudlets based on their execution length.
- Run the CloudSim simulation.
- Observe and verify the execution order of the cloudlets.

CLOUDSIM

CloudSim is a simulation framework that enables researchers and developers to model and experiment with cloud computing environments without requiring actual large-scale cloud infrastructure.

It can be used to simulate datacenters, virtual machines, cloudlets, resource allocation and scheduling policies. In this experiment, the main components are the Virtual Machine, Cloudlet and Datacenter Broker.

SHORTEST JOB FIRST SCHEDULING

Shortest Job First is a scheduling technique in which the task having the smallest execution time is selected before longer tasks. In this experiment, the cloudlets are sorted in ascending order of their execution length.

Cloudlet 0 → 40000 MI

Cloudlet 1 → 10000 MI

Cloudlet 2 → 20000 MI

SJF order:

Cloudlet 1 → Cloudlet 2 → Cloudlet 0

PROCEDURE

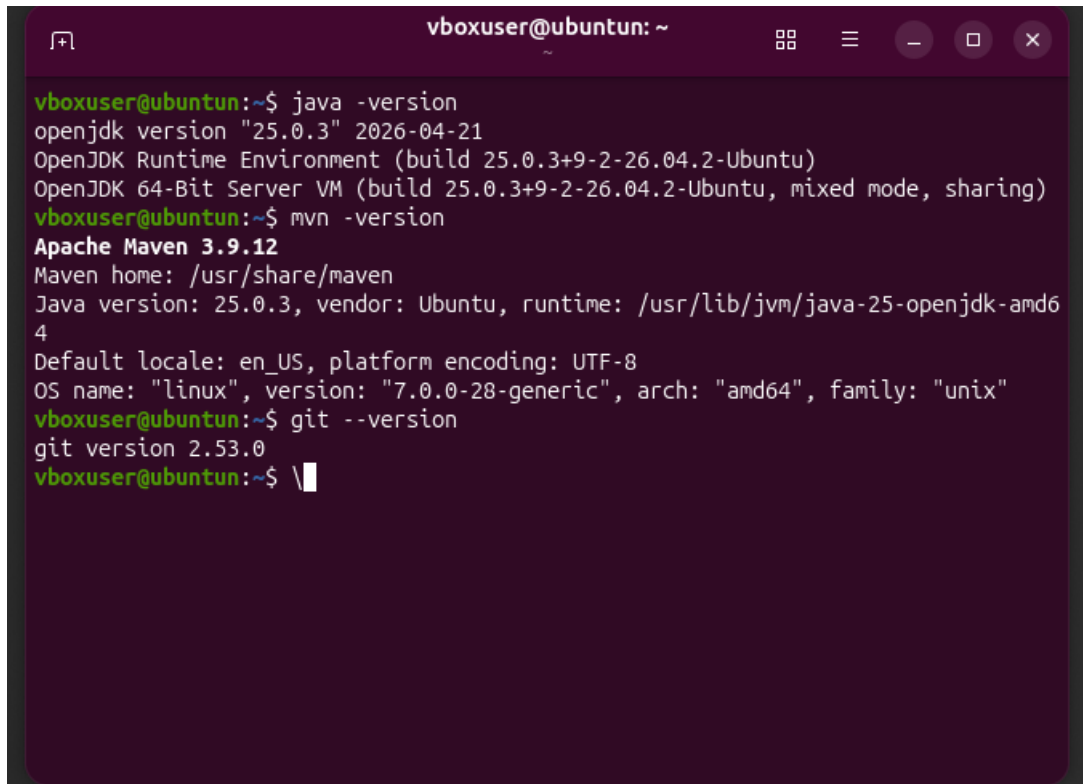
Step 1: Install the Required Software

The required Java development environment, Git and Maven were installed in Ubuntu.

```
sudo apt update
sudo apt install default-jdk git maven -y
```

The installations were verified using:

```
java -version
mvn -version
git -version
```

A terminal window titled 'vboxuser@ubuntu: ~' with standard window controls. It shows the output of three commands: 'java -version' displays 'openjdk version "25.0.3" 2026-04-21', 'mvn -version' displays 'Apache Maven 3.9.12' and other details, and 'git --version' displays 'git version 2.53.0'.

```
vboxuser@ubuntu:~$ java -version
openjdk version "25.0.3" 2026-04-21
OpenJDK Runtime Environment (build 25.0.3+9-2-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 25.0.3+9-2-26.04.2-Ubuntu, mixed mode, sharing)
vboxuser@ubuntu:~$ mvn -version
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 25.0.3, vendor: Ubuntu, runtime: /usr/lib/jvm/java-25-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-28-generic", arch: "amd64", family: "unix"
vboxuser@ubuntu:~$ git --version
git version 2.53.0
vboxuser@ubuntu:~$ \
```

Step 2: Configure Java Environment

Java 21 was selected as the active Java version for the CloudSim project. The Java runtime and compiler configurations were checked using the update-alternatives command.

```
sudo update-alternatives --config java
sudo update-alternatives --config javac

java -version
javac -version
```

```
vboxuser@ubuntu: ~/cloudsim
ue

Press <enter> to keep the current choice[*], or type selection number: 1
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bin/java to provide /usr/bin/java (java) in manual mode
vboxuser@ubuntu:~$ sudo update-alternatives --config javac
There are 2 choices for the alternative javac (providing /usr/bin/javac).

  Selection    Path
  -----
* 0            /usr/lib/jvm/java-25-openjdk-amd64/bin/javac    2511    auto mode
1             /usr/lib/jvm/java-21-openjdk-amd64/bin/javac    2111    manual mode
2             /usr/lib/jvm/java-25-openjdk-amd64/bin/javac    2511    manual mode

Press <enter> to keep the current choice[*], or type selection number: 1
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bin/javac to provide /usr/bin/javac (javac) in manual mode
vboxuser@ubuntu:~$ java -version
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-26.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-26.04-Ubuntu, mixed mode, sharing)
vboxuser@ubuntu:~$ cd ~
```

Step 3: Clone the CloudSim Repository

The official CloudSim repository was cloned using Git.

```
cd ~
git clone https://github.com/Cloudslab/cloudsim.git
cd cloudsim
```

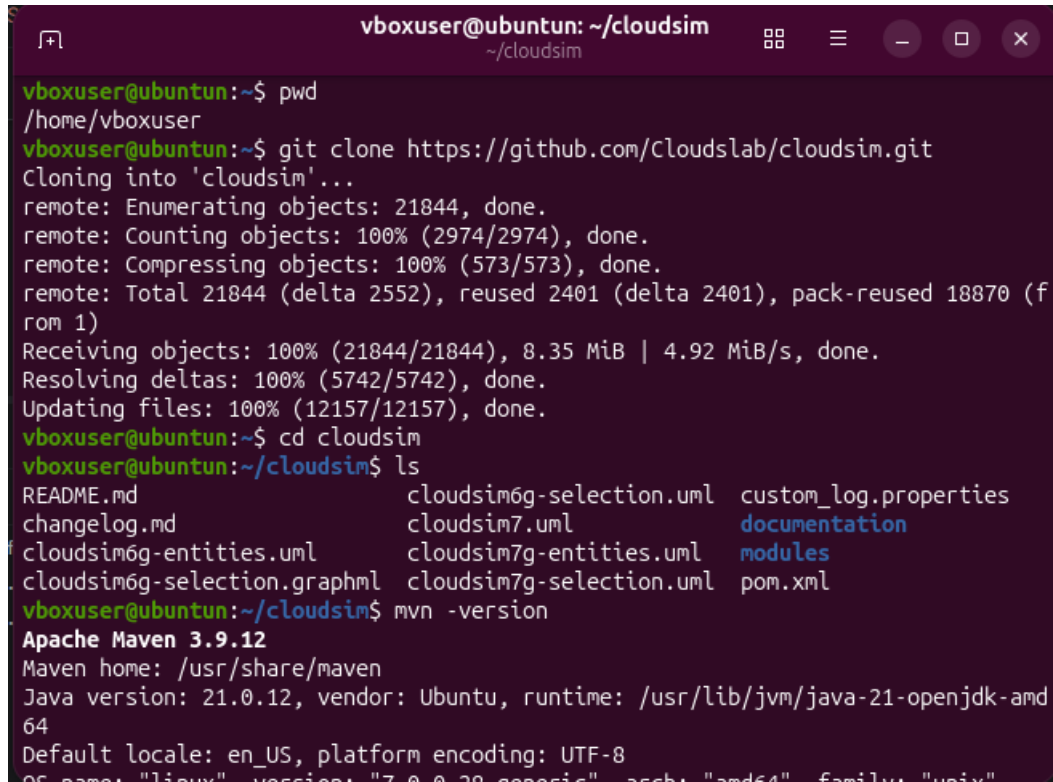
```
vboxuser@ubuntu: ~/cloudsim
vboxuser@ubuntu:~$ pwd
/home/vboxuser
vboxuser@ubuntu:~$ git clone https://github.com/Cloudslab/cloudsim.git
Cloning into 'cloudsim'...
remote: Enumerating objects: 21844, done.
remote: Counting objects: 100% (2974/2974), done.
remote: Compressing objects: 100% (573/573), done.
remote: Total 21844 (delta 2552), reused 2401 (delta 2401), pack-reused 18870 (from 1)
Receiving objects: 100% (21844/21844), 8.35 MiB | 4.92 MiB/s, done.
Resolving deltas: 100% (5742/5742), done.
Updating files: 100% (12157/12157), done.
vboxuser@ubuntu:~$ cd cloudsim
vboxuser@ubuntu:~/cloudsim$ ls
README.md          cloudsimg-selection.uml  custom_log.properties
changelog.md       cloudsimg7.uml          documentation
cloudsim6g-entities.uml  cloudsimg7g-entities.uml  modules
cloudsim6g-selection.graphml  cloudsimg7g-selection.uml  pom.xml
vboxuser@ubuntu:~/cloudsim$ mvn -version
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 21.0.12, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-28-generic", arch: "amd64", family: "unix"
```

Step 4: Build the CloudSim Project

The CloudSim project was compiled and its dependencies were installed using Maven.

```
mvn clean install
```

The build completed successfully with the CloudSim modules reporting SUCCESS and BUILD SUCCESS.

A terminal window titled 'vboxuser@ubuntun: ~/cloudsim' showing the execution of Maven commands. The user first runs 'pwd' and then 'git clone https://github.com/Cloudslab/cloudsim.git'. The output shows the repository being cloned into a directory named 'cloudsim'. After navigating to the 'cloudsim' directory with 'cd cloudsim', the user runs 'ls' to list the files, which include README.md, changelog.md, various .uml files, pom.xml, and documentation modules. Finally, the user runs 'mvn -version', which outputs 'Apache Maven 3.9.12' and other system details like Java version and default locale.

```
vboxuser@ubuntun:~$ pwd
/home/vboxuser
vboxuser@ubuntun:~$ git clone https://github.com/Cloudslab/cloudsim.git
Cloning into 'cloudsim'...
remote: Enumerating objects: 21844, done.
remote: Counting objects: 100% (2974/2974), done.
remote: Compressing objects: 100% (573/573), done.
remote: Total 21844 (delta 2552), reused 2401 (delta 2401), pack-reused 18870 (from 1)
Receiving objects: 100% (21844/21844), 8.35 MiB | 4.92 MiB/s, done.
Resolving deltas: 100% (5742/5742), done.
Updating files: 100% (12157/12157), done.
vboxuser@ubuntun:~$ cd cloudsim
vboxuser@ubuntun:~/cloudsim$ ls
README.md          cloudsimg-selection.uml  custom_log.properties
changelog.md       cloudsimg7.uml          documentation
cloudsim6g-entities.uml  cloudsimg7g-entities.uml  modules
cloudsim6g-selection.graphml  cloudsimg7g-selection.uml  pom.xml
vboxuser@ubuntun:~/cloudsim$ mvn -version
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 21.0.12, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-28-generic", arch: "amd64", family: "unix"
```

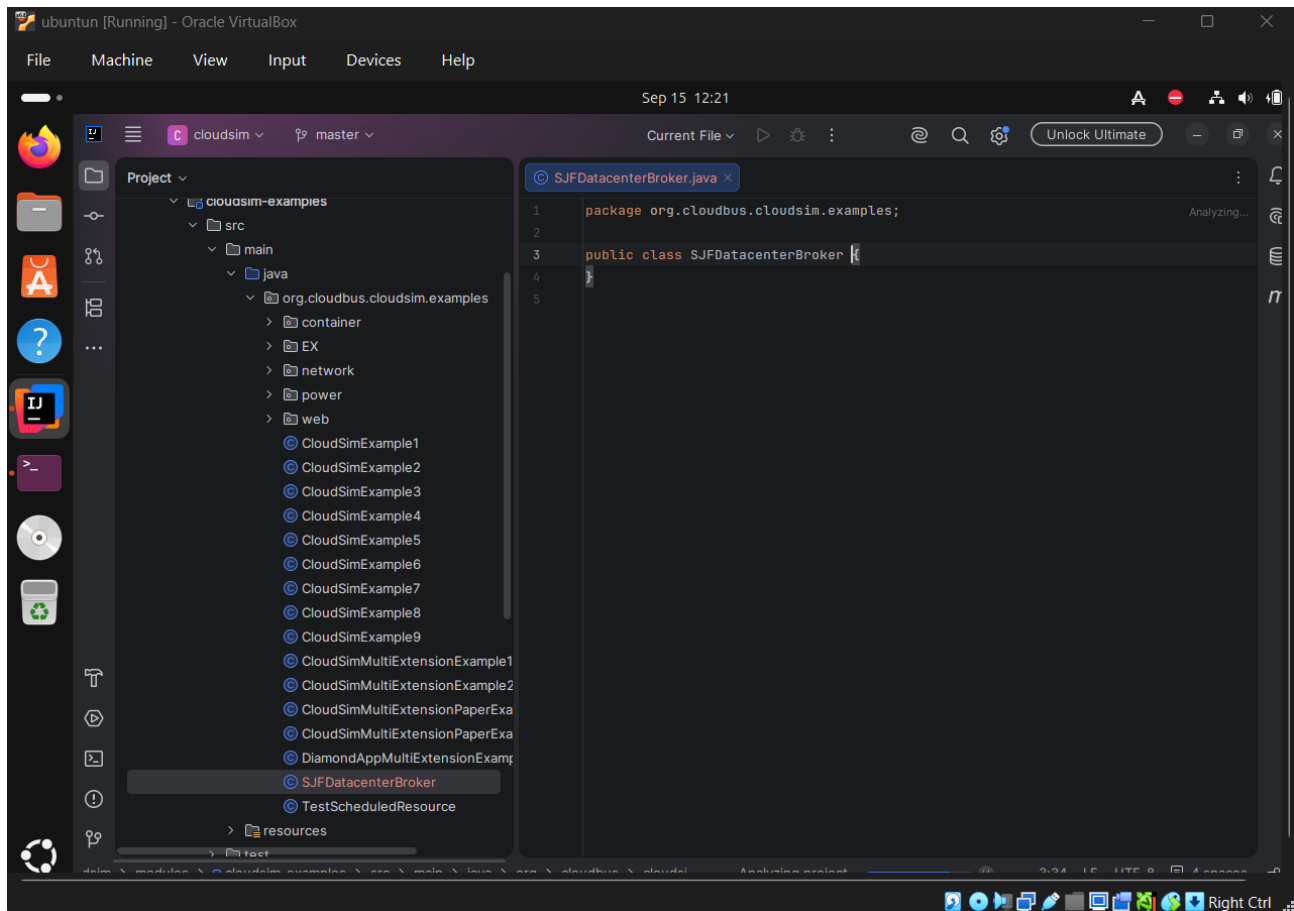
Step 5: Import CloudSim into IntelliJ IDEA

The cloned CloudSim directory was opened in IntelliJ IDEA as a Maven project. IntelliJ loaded the project structure and required dependencies.

```
/home/vboxuser/cloudsim
```

The relevant project path used for the experiment was:

```
cloudsim
├── modules
│   └── cloudsim-examples
│       ├── src
│       │   ├── main
│       │   │   └── java
│       │   │       └── org.cloudbus.cloudsim.examples
```



Step 6: Create the Custom SJF Broker

A new Java class named `SJFDatacenterBroker.java` was created inside the `org.cloudbus.cloudsim.examples` package. The class extends the `CloudSimDatacenterBroker` class and overrides `submitCloudlets()` to sort cloudlets according to their execution length.

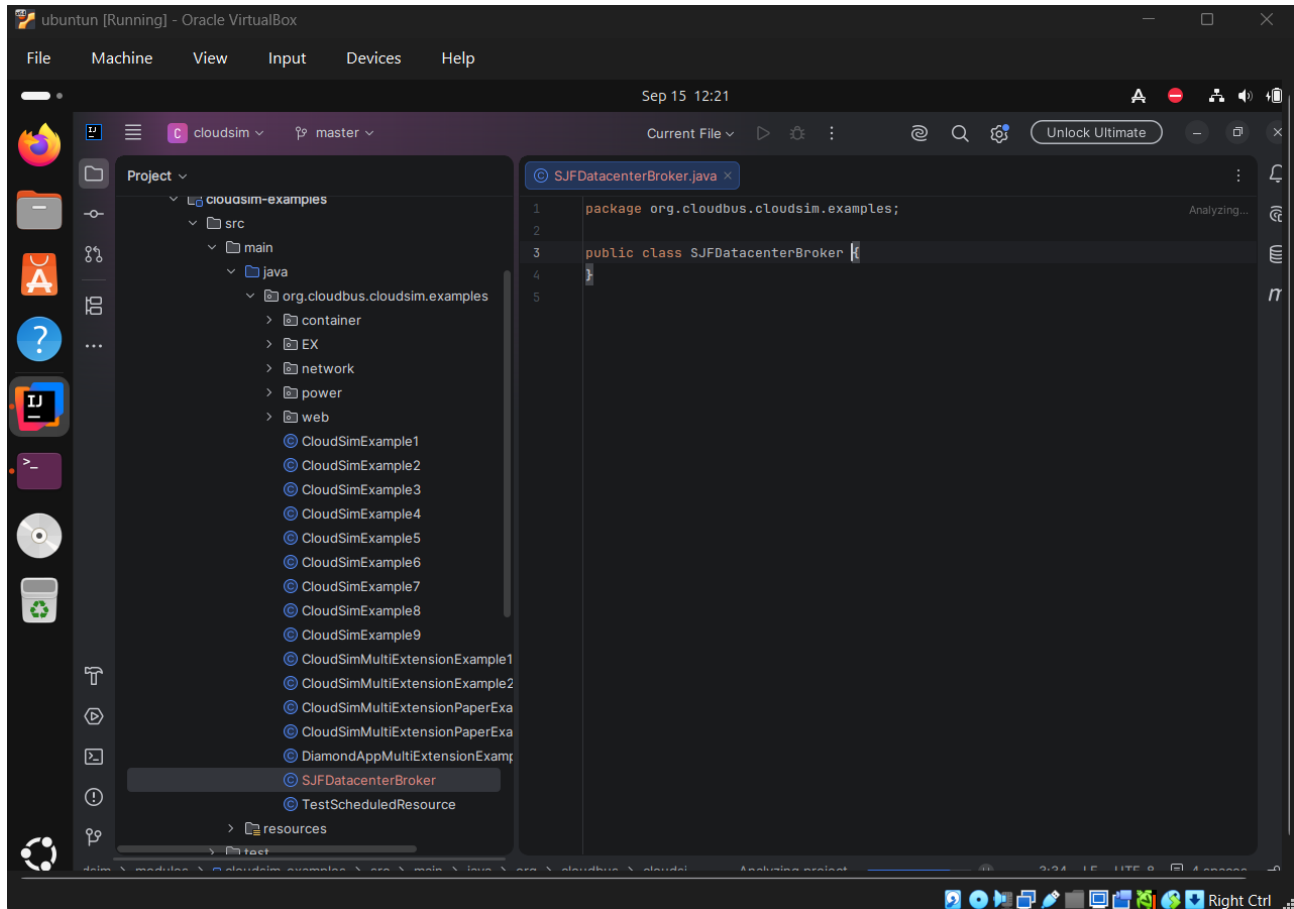
```
Collections.sort(
    getCloudletList(),
    new Comparator<Cloudlet>() {
        @Override
        public int compare(Cloudlet c1, Cloudlet c2) {
            return Long.compare(
```

```

        c1.getCloudletLength(),
        c2.getCloudletLength()
    );
}
}
);

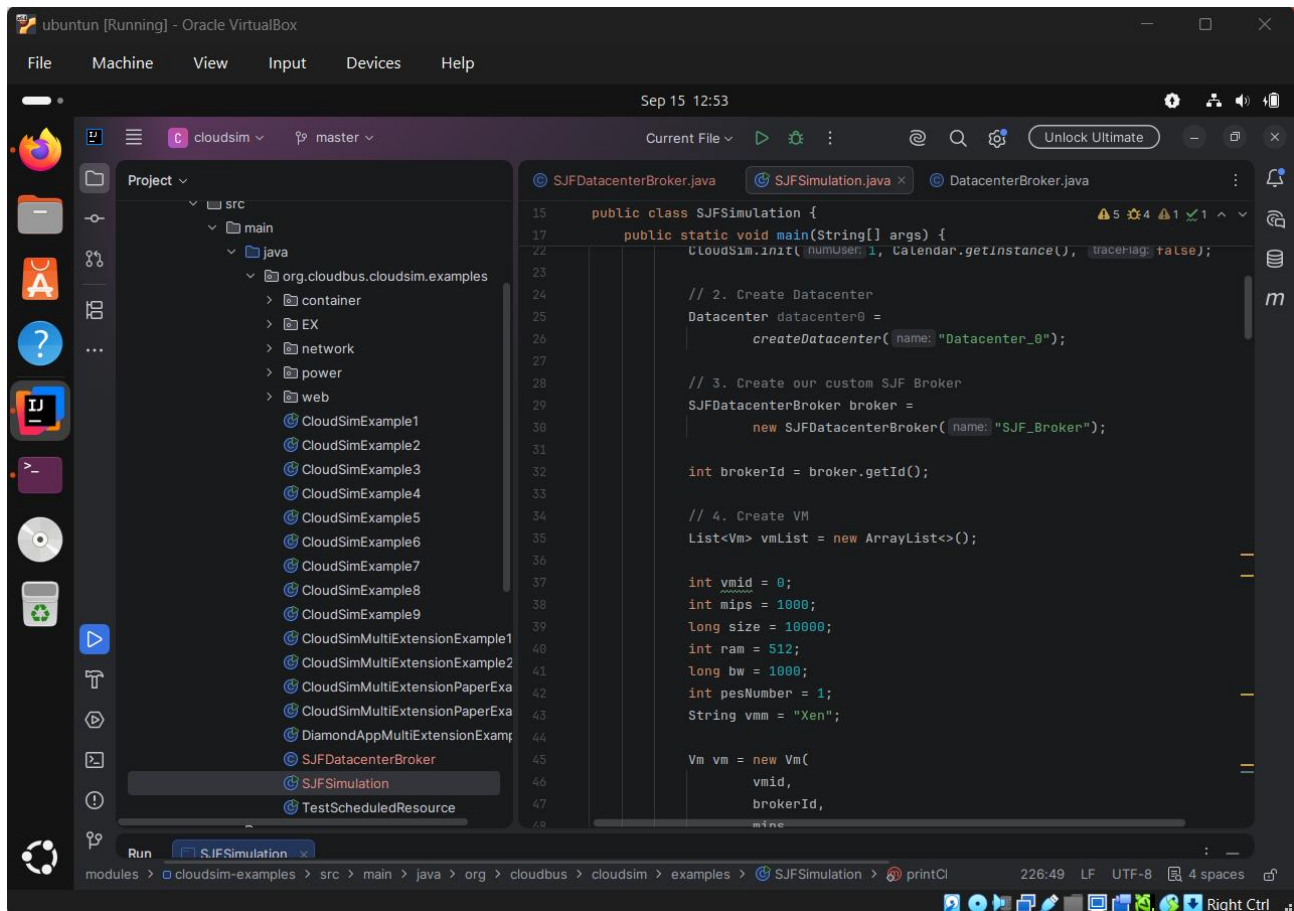
super.submitCloudlets();

```



Step 7: Create the Main Simulation Class

A new Java class named `SJFSimulation.java` was created in the same package. The main simulation initializes `CloudSim`, creates the datacenter and broker, creates the virtual machine and cloudlets, starts the simulation, retrieves the results and prints the output.



Step 8: Create Cloudlets

Three cloudlets with different execution lengths were created:

Cloudlet ID	Execution Length
Cloudlet 0	40,000 MI
Cloudlet 1	10,000 MI
Cloudlet 2	20,000 MI

The cloudlets were intentionally added in the order Cloudlet 0, Cloudlet 1 and Cloudlet 2.

Step 9: Apply SJF Scheduling

The custom broker examined the execution length of each pending cloudlet and sorted them in ascending order.

Before sorting:

Cloudlet 0 → 40000 MI
 Cloudlet 1 → 10000 MI
 Cloudlet 2 → 20000 MI

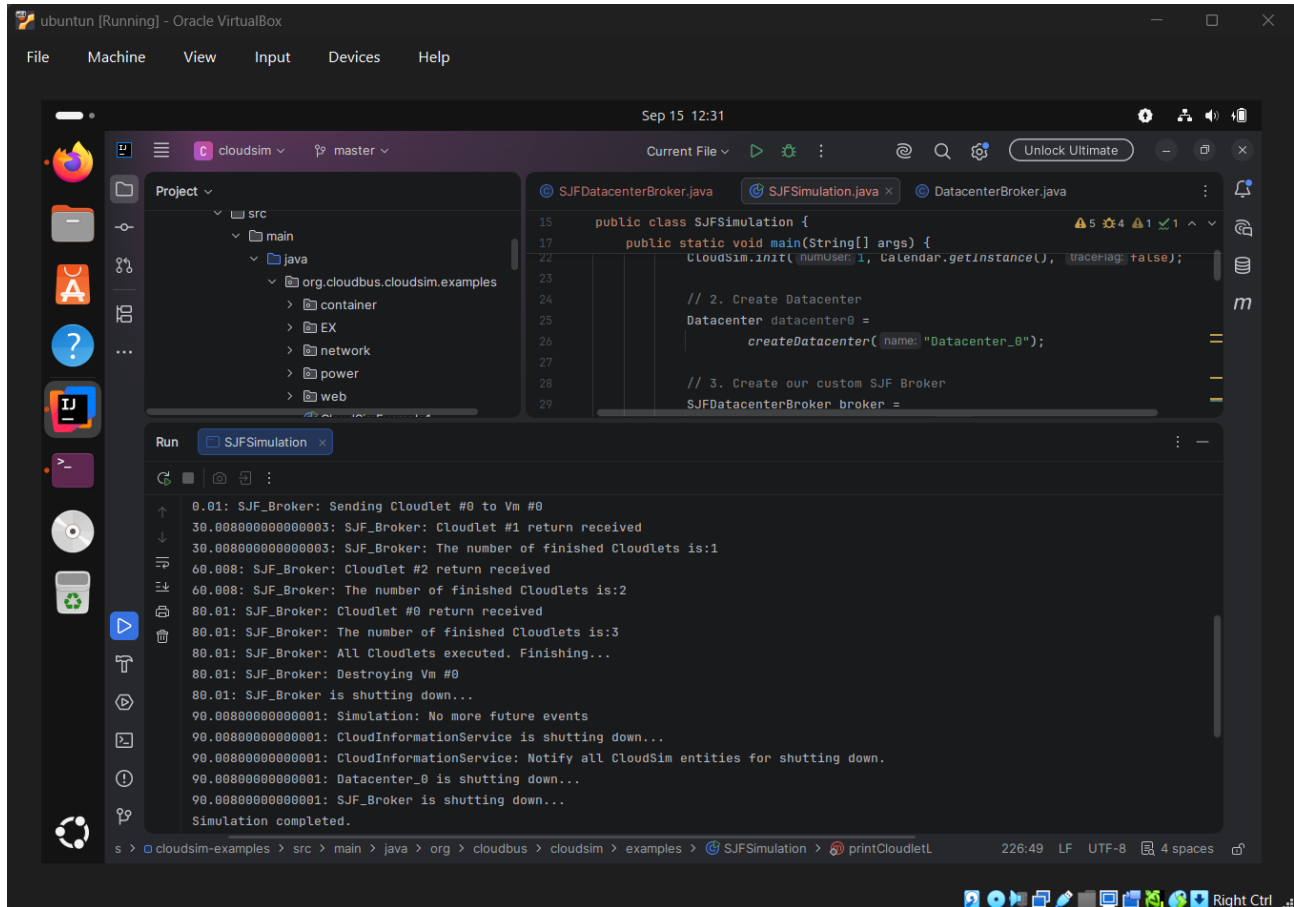
After SJF sorting:

Cloudlet 1 → 10000 MI

Cloudlet 2 → 20000 MI
Cloudlet 0 → 40000 MI

Step 10: Run the Simulation

The SJFSimulation.java class was executed from IntelliJ IDEA using the Run SJFSimulation.main() option. The console output was examined to verify the execution order.



OUTPUT

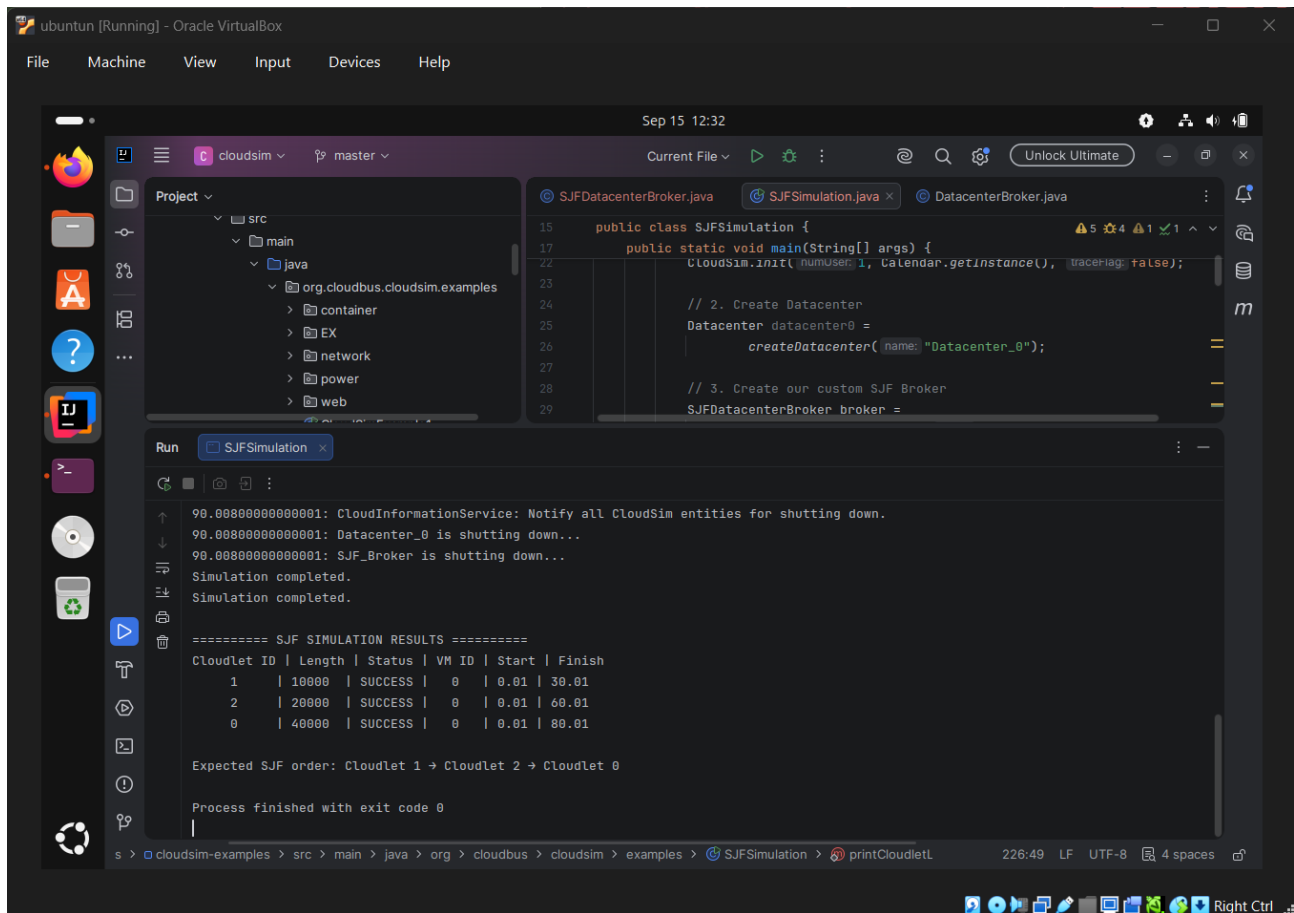
The following output was obtained from the simulation:

===== SJF SIMULATION RESULTS =====

Cloudlet ID	Length	Status	VM ID	Start	Finish
1	10000	SUCCESS	0	0.01	30.01
2	20000	SUCCESS	0	0.01	60.01
0	40000	SUCCESS	0	0.01	80.01

Expected SJF order: Cloudlet 1 → Cloudlet 2 → Cloudlet 0

Process finished with exit code 0



RESULT

Thus, a custom Shortest Job First (SJF) scheduling algorithm was successfully implemented in CloudSim by extending the DatacenterBroker and modifying the cloudlet submission process.

The simulation successfully executed all three cloudlets, with the completion order:

Cloudlet 1 → Cloudlet 2 → Cloudlet 0

The results confirm that the shortest cloudlet was completed first, followed by the medium and longest cloudlets. Hence, the Shortest Job First scheduling logic was successfully implemented and verified.

CONCLUSION

The experiment demonstrated the implementation of a custom scheduling policy in CloudSim. A custom SJFDatacenterBroker was developed to sort cloudlets based on their execution length before submission.

The successful execution of Cloudlet 1, Cloudlet 2 and Cloudlet 0 in increasing order of their lengths verified the correctness of the SJF scheduling algorithm. The experiment also demonstrated how CloudSim can be extended to develop and evaluate different scheduling and resource-management strategies in a simulated cloud environment.